AERO-VPA: FEW-STEP FLOW-MATCHING CONTROL OVER NON-RECONSTRUCTIVE JEPA LATENTS FOR AGILE FLIGHT ON EDGE HARDWARE

Yeonseok Lee
SLING AI Inc.
Incheon, Republic of Korea
ylee@sling.ai.kr

ABSTRACT

Vision-Language-Action (VLA) models generalize across tasks, but their autoregressive decoders require a number of sequential forward passes that grows with the action horizon, and on size-, weight-, and power-constrained aerial platforms each pass is bounded below by the time needed to stream billions of parameters from memory. We present Aero-VPA, a lightweight Vision-Predictor-Action architecture for agile three-dimensional navigation on edge hardware. A compact Vision Transformer embeds RGB-D observations into a latent space shaped by a non-reconstructive Joint-Embedding Predictive Architecture (JEPA) world model, trained with variance–covariance regularization instead of pixel reconstruction. A discrete primitive selector and an ensemble latent predictor, conditioned on the active skill and the executing command, forecast the next latent state, and a Conditional Flow Matching (CFM) solver generates a complete chunk of velocity and yaw-rate setpoints in two Euler steps, so that every decision requires five sequential network evaluations irrespective of the horizon length. The normalized dispersion of the predictor ensemble provides an epistemic uncertainty signal that requires no future observation; it attenuates translational velocity, shortens the validity of each chunk, and triggers a minimum-jerk transition to hover, while a model-free braking envelope limits speed to the measured free space. We specify an asynchronous deployment on an NVIDIA Jetson Orin Nano (8 GB, 15 W) that streams interpolated setpoints to a PX4 flight controller over micro-XRCE-DDS, together with a software- and hardware-in-the-loop benchmark protocol against classical, reinforcement-learning, and generative baselines, with hypotheses, metrics, and ablations fixed in advance. No experimental results are reported.

Keywords Aerial Robotics · Joint-Embedding Predictive Architecture · Flow Matching · Edge Computing · Safe Learning-Based Control

1 Introduction

Navigating cluttered three-dimensional environments at speed requires a micro aerial vehicle (MAV) to convert onboard sensing into motor commands within a fraction of the time it needs to reach the nearest obstacle. Modular pipelines that chain visual-inertial odometry, volumetric mapping, and trajectory optimization [1–4] remain the dominant engineering solution, but each stage adds latency and couples the system to the accuracy of an explicit map; learned sensorimotor policies have accordingly been shown to fly faster and more robustly in the wild [5] and to reach champion-level performance in drone racing [6, 7]. In parallel, Vision-Language-Action (VLA) models [8–10] have demonstrated broad semantic generalization by casting control as sequence modeling. Most VLAs, however, emit actions through an autoregressive language-model decoder: an H-step chunk of d_a-dimensional actions discretized into q tokens per component requires $N = qHd_a$ dependent forward passes. At batch size one, every pass must stream the decoder weights from memory, so the latency is bounded below by $N|W_{\mathrm{LLM}}|b/BW$, where $|W_{\mathrm{LLM}}|$ is the number of parameters, b the bytes per parameter, and BW the memory bandwidth. On an NVIDIA Jetson Orin Nano 8 GB, whose LPDDR5 interface provides 68 GB/s in its original configuration and 102 GB/s in the JetPack 6.2 “Super” configurations [11], a

3B-parameter model in 16-bit precision (6 GB, three quarters of the device memory) needs at least 88 ms per pass at 68 GB/s. A single four-component velocity command ($q = 1$, $H = 1$, $d_a = 4$) then takes at least 0.35 s (at most 2.8 Hz), and a 16-step chunk at least 5.6 s, during which a vehicle flying at 5 m/s travels 28 m. These bounds are optimistic: they ignore compute, attention over the context, and memory shared with perception and middleware.

The Vision-Predictor-Action (VPA) framework [12] removes the language decoder from the control loop. A compact encoder maps observations into a latent space learned by a Joint-Embedding Predictive Architecture (JEPA) [13–15], which predicts future *embeddings* rather than future pixels and is therefore not required to model task-irrelevant appearance; a predictor forecasts the latent consequence of the selected skill; and a Conditional Flow Matching (CFM) solver [16–18] transports Gaussian noise to an entire action chunk by integrating a learned ordinary differential equation (ODE). Denoising diffusion policies [19] generate chunks by iterative denoising, commonly with 10 to 100 network evaluations per chunk; the CFM sampler is instead a deterministic ODE map from a sampled initial condition, and straight transport paths admit accurate integration in very few steps; the number of sequential network evaluations per decision is $K + 3$ for K integration steps, independent of the horizon H [12]. A reference implementation with fewer than 20M trainable parameters ran a decision step in 10.5–14.2 ms on an Apple M4 and succeeded in 40.0% of LIBERO-10 manipulation episodes [12]. Flight, however, differs from tabletop manipulation in three respects that this paper addresses: the platform is underactuated and cannot pause in free space, so the controller needs a safe action whenever its predictions become unreliable; commands must be streamed to a flight controller at a fixed rate under strict failsafe timing [20]; and the compute budget is that of an embedded module rather than a workstation.

This paper formalizes Aero-VPA, a specialization of VPA to agile aerial navigation, and specifies the protocol by which it will be evaluated. Our contributions are:

- **Aero-VPA formulation.** A goal-conditioned partially observable formulation with chunks of heading-frame velocity and yaw-rate setpoints $A_t = [v_x, v_y, v_z, \dot{\psi}]_{t:t+H-1}$; an RGB-D Vision Transformer encoder trained jointly with a primitive- and command-conditioned JEPA predictor under VICReg regularization [21]; and a CFM solver sampled with $K = 2$ Euler steps, giving a sequential depth of five network evaluations per decision. We add a noise-persistence rule and prove a Lipschitz bound on the change between consecutive chunks that it enables (Proposition 3.1).
- **Latent-uncertainty safety layer.** An epistemic uncertainty score computed from the normalized dispersion of a predictor ensemble [22] without access to future observations, an asymmetric filter, a monotone velocity-attenuation law $\alpha(\bar{\sigma})$, an uncertainty-dependent chunk validity horizon, and a minimum-jerk braking transition to hover. A model-free braking envelope complements the learned signal, and we state the assumptions under which it guarantees a stop within the measured free space (Proposition 3.2).
- **Asynchronous edge deployment.** A two-rate architecture for the Jetson Orin Nano (8 GB, 15 W) and a PX4 flight controller connected through micro-XRCE-DDS [23], comprising latency-compensated chunk indexing, continuous chunk blending, minimum-acceleration setpoint interpolation, an analytic parameter and memory-bandwidth budget, and an adaptation to quadruped robots.
- **Benchmark protocol.** A software-in-the-loop (SITL) and hardware-in-the-loop (HITL) protocol with baselines, metric definitions, falsifiable hypotheses with acceptance criteria, a statistical analysis plan, and an ablation matrix, all fixed before any experiment is executed. This paper reports no experimental results.

2 Related Work

2.1 Visual Predictive World Models and Non-Reconstructive Latents

Generative world models learn dynamics by reconstructing observations and have achieved strong results in reinforcement learning [24], but reconstruction obliges the model to account for every pixel, including illumination, texture, and sensor noise that are irrelevant to control. Joint-embedding predictive architectures [13] instead predict the embedding of a target from the embedding of a context: I-JEPA [14] predicts masked image regions in representation space, V-JEPA [15] extends this to video, and V-JEPA 2 [25] shows that an action-conditioned latent predictor post-trained on robot interaction data supports zero-shot manipulation through planning in representation space, at the cost of iterative optimization at inference time. TD-MPC2 [26] likewise learns a decoder-free latent model for continuous control. Without a reconstruction target, joint-embedding objectives admit collapsed solutions; VICReg [21] prevents them with variance and covariance penalties on the embedding batch. In robot learning, ACT-JEPA [27] uses latent prediction to regularize imitation policies, VLA-JEPA [28] adds a latent world model to a VLA backbone, and VPA [12, 29] couples a VICReg-regularized JEPA predictor with a discrete skill selector and a flow-matching action head without a language decoder. Aero-VPA retains this structure and adapts its inputs, conditioning, and safety logic to flight.

2.2 Continuous Normalizing Flows and Flow Matching for Robot Action Generation

Continuous normalizing flows define a generative model by integrating a learned vector field [30]. Flow matching [16], rectified flow [17], and stochastic interpolants [18] train such fields by regression onto the velocity of a prescribed probability path, avoiding simulation during training; straight paths permit few-step integration. In robotics, action chunking with transformers (ACT) [31] showed that predicting multi-step chunks mitigates compounding error, Diffusion Policy [19] models multimodal chunk distributions by iterative denoising, and consistency distillation reduces the number of denoising steps [32]. π_0 [10] attaches a flow-matching action expert to a multi-billion-parameter vision-language model, and Octo [33] attaches a diffusion head to a transformer backbone of 27M–93M parameters. Real-time chunking [34] addresses the discontinuities that arise when chunks are replanned asynchronously by inpainting the portion of the next chunk that overlaps the current one. Aero-VPA uses a CFM head with $K = 2$, conditions it on forecast latents rather than on the hidden states of a language backbone, and addresses inter-chunk consistency by noise persistence and continuous blending (Section 3.4).

Closest to our work is Qantara [35], which also couples a JEPA latent model with flow matching on actions. Qantara trains its encoder and a shared block-causal transformer predictor jointly from scratch: the next latent is learned with a Brownian-bridge objective anchored at the current latent, the action with a noise-to-data flow-matching objective, and SIGReg serves as the only anti-collapse term. By sampling separate noise levels for the action and latent axes, (τ^a, τ^z) in their notation, along the edges and the diagonal of the unit square, a single 21M-parameter checkpoint serves latent planning with the cross-entropy method, behaviour cloning, and a video-inverse path that predicts the next latent and then infers the connecting action. Aero-VPA operates at a similar scale, also samples actions with two Euler steps, and follows the same predict-then-act chain as that last path, but resolves the design trade-off differently: it commits to a single goal- and primitive-conditioned inference path and decouples its components. The VICReg-regularized encoder and ensemble predictor are trained first and frozen, which fixes the latent scale that normalizes the uncertainty signal of Section 3.5. The CFM head is trained afterwards on velocity chunks, so it can be retrained per embodiment and compiled as an independent inference engine. Qantara is evaluated in simulated navigation, manipulation, and reaching tasks, with timing measured on an H100 GPU. Aero-VPA instead targets agile three-dimensional flight under the fixed-rate setpoint interface of PX4 on a 15 W embedded module, and couples action generation to an uncertainty-gated safety layer.

2.3 Edge Autonomous Flight and Computational Constraints in Aerial and Legged Systems

Classical agile-flight stacks combine visual-inertial state estimation [1, 36], occupancy mapping [2], and polynomial or gradient-based planning [3, 4], and have been scaled to swarms in unstructured environments [37]. Learning-based approaches replace part of this stack: Loquercio et al. [5] trained a depth-based policy by imitating a privileged planner and executed it on an embedded GPU, Kaufmann et al. [6] trained racing policies with deep reinforcement learning, and Agilicious [38] provides open hardware and software with onboard GPU compute for vision-based agile flight. Learned residual dynamics address aerodynamic effects such as ground effect [39] and strong wind [40]. General navigation models such as GNM [41] and NoMaD [42] transfer across ground robots but target lower speeds. In legged locomotion, hierarchical designs pair a high-level command policy with a low-level controller trained by reinforcement learning [43–45]; this separation of time scales motivates the cross-embodiment discussion in Section 4.5. Across these platforms the binding constraints are size, weight, and power (SWaP), memory bandwidth, and the fixed-rate interfaces of the low-level controller [20, 46], which together determine the design choices of Section 4.

3 Problem Formulation and Proposed Architecture

3.1 Goal-Conditioned POMDP with Velocity Chunks

We model goal-directed flight as a goal-conditioned partially observable Markov decision process (POMDP)

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \Omega, \mathcal{G}, R, \gamma_{\text{disc}}), \tag{1}$$

whose components are defined as follows. A state $s = (p, v, \mathbf{R}, \omega, w, \mathcal{X}) \in \mathcal{S}$ comprises the position $p \in \mathbb{R}^3$ and velocity $v \in \mathbb{R}^3$ of the vehicle in a local north-east-down (NED) world frame $\mathcal{W}$, its attitude $\mathbf{R} \in SO(3)$, its body angular rate $\omega \in \mathbb{R}^3$, the ambient wind $w \in \mathbb{R}^3$, and the occupied set $\mathcal{X} \subset \mathbb{R}^3$ of the environment. The vehicle observes $o_t = (I_t, S_t) \in \mathcal{O}$ at decision step t, captured at wall-clock time τ_t, where $I_t \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times 4}$ is an RGB-D frame ($H_{\text{img}} = 128$, $W_{\text{img}} = 160$) and

$$S_t = \left(\hat{v}_t^{\mathcal{H}},\, \hat{n}_t^{\mathcal{B}},\, \hat{\omega}_t^{\mathcal{B}},\, \bar{a}_t\right) \in \mathbb{R}^{d_s}, \qquad d_s = 13, \tag{2}$$

is the proprioceptive state estimated by the PX4 EKF2: the velocity $\hat{v}_t^{\mathcal{H}} \in \mathbb{R}^3$ expressed in the heading frame $\mathcal{H}_t$ (the gravity-aligned forward-right-down frame obtained by rotating $\mathcal{W}$ about its down axis by the estimated yaw $\hat{\psi}_t$), the

unit gravity direction $\hat{n}_t^{\mathcal{B}} \in \mathbb{S}^2$ in the body frame $\mathcal{B}$, which encodes roll and pitch, the body rates $\hat{\omega}_t^{\mathcal{B}} \in \mathbb{R}^3$, and the setpoint $\bar{a}_t \in \mathbb{R}^4$ dispatched most recently to the flight controller, expressed in $\mathcal{H}_t$. The observation model Ω is induced by image formation, $I_t = \mathcal{I}(s(\tau_t), \eta_t)$, where η_t collects nuisance factors such as illumination, texture, motion blur, and depth noise.

A mission $G = (x, (w^{(m)})_{m=1}^{M}) \in \mathcal{G}$ consists of an optional natural-language instruction x and an ordered sequence of M waypoints $w^{(m)} \in \mathbb{R}^3$ in $\mathcal{W}$. With the active waypoint index m_t (Section 3.3) and the estimated position $\hat{p}_t$, the policy receives the goal feature

$$g_t = \left(\frac{r_t}{\max(\|r_t\|_2, \epsilon_r)}, \frac{\min(\|r_t\|_2, D_{\max})}{D_{\max}} \right) \in \mathbb{R}^4, \qquad r_t = \mathbf{R}_z(\hat{\psi}_t)^\top \big(w^{(m_t)} - \hat{p}_t \big), \tag{3}$$

where $\mathbf{R}_z(\psi) \in SO(3)$ is the rotation about the down axis by ψ, $D_{\max}$ is a saturation distance, and $\epsilon_r > 0$ avoids division by zero.

The per-step action $a = (v_x, v_y, v_z, \dot{\psi}) \in \mathcal{A}_0 = [-\bar{v}_h, \bar{v}_h]^2 \times [-\bar{v}_z, \bar{v}_z] \times [-\bar{\dot{\psi}}, \bar{\dot{\psi}}]$ is a velocity setpoint and a yaw-rate setpoint, with speed limits $\bar{v}_h$, $\bar{v}_z$, and $\bar{\dot{\psi}}$. At each decision step the controller outputs an open-loop chunk of H knots spaced Δ seconds apart,

$$A_t = [v_x, v_y, v_z, \dot{\psi}]_{t:t+H-1} = \big[a_{t,0};\, a_{t,1};\, \ldots;\, a_{t,H-1} \big] \in \mathcal{A}_0^H \subset \mathbb{R}^{H \times d_a}, \qquad d_a = 4, \tag{4}$$

where knot $a_{t,k}$ applies at time $\tau_t + k\Delta$ and its velocity components are expressed in the heading frame $\mathcal{H}_t$ frozen at the capture time τ_t. We use $\Delta = 20\,\text{ms}$ and $H = 16$, so that a chunk spans 0.32 s. The transition kernel $\mathcal{T}(s' \mid s, a)$ is induced by the PX4 velocity and attitude controllers, the rigid-body and rotor dynamics, and the wind. The reward

$$R(s, G; m) = -\left\| p - w^{(m)} \right\|_2 - \lambda_{\text{col}} \mathbb{1}[\text{dist}(p, \mathcal{X}) < r_v], \tag{5}$$

with vehicle radius r_v and collision weight $\lambda_{\text{col}} > 0$, and the discount $\gamma_{\text{disc}} \in [0, 1)$ define the task for analysis and for the reinforcement-learning baselines of Section 5.3. Aero-VPA itself is trained by self-supervised latent prediction and by imitation of a privileged expert (Section 3.8).

3.2 Non-Reconstructive JEPA World Model

A Vision Transformer [47] E_ψ with patch size 16 divides the four-channel frame into $N_p = (128/16)(160/16) = 80$ patches, processes them together with a class token through six blocks of width 384, and projects the class token to the latent

$$z_t = E_\psi(I_t) \in \mathcal{Z} \subset \mathbb{R}^d, \qquad d = 256. \tag{6}$$

The depth channel is clipped to the sensor range and normalized before patch embedding. When an instruction x is given, a frozen CLIP text encoder [48] computes $c_{\text{text}} \in \mathbb{R}^{512}$ once per mission; otherwise c_{text} is a learned null embedding.

The predictor P_ω forecasts the latent state $T_\nu = 0.1\,\text{s}$ ahead, conditioned on the current latent, the active primitive $u_t \in \mathcal{U}$ (Section 3.3), and the executing setpoint $\bar{a}_t$. It consists of $N_e = 5$ independently initialized heads $P_\omega^{(i)}$:

$$\hat{z}_{t+1}^{(i)} = P_\omega^{(i)}\big(z_t, \text{Embed}(u_t), \bar{a}_t\big), \qquad \hat{z}_{t+1} = \frac{1}{N_e} \sum_{i=1}^{N_e} \hat{z}_{t+1}^{(i)}, \tag{7}$$

where $\text{Embed} : \mathcal{U} \to \mathbb{R}^{d_u}$ is a learned primitive embedding ($d_u = 32$) and the subscript $t+1$ abbreviates the time $\tau_t + T_\nu$. Conditioning on $\bar{a}_t$, which is known before the decision, makes the predictor action-conditioned without placing the solver output on its input, so the dependency chain of a decision step remains acyclic. The encoder and predictor are trained jointly with

$$\mathcal{L}_{\text{JEPA}}(\psi, \omega) = \mathbb{E}_t \left[\frac{1}{N_e} \sum_{i=1}^{N_e} \left\| \hat{z}_{t+1}^{(i)} - \text{sg}\big(E_{\bar{\psi}}(I_{t+1})\big) \right\|_2^2 \right] + \lambda_v\, v(Z) + \lambda_c\, c(Z), \tag{8}$$

where I_{t+1} is the frame captured closest to $\tau_t + T_\nu$, $\text{sg}(\cdot)$ is the stop-gradient operator, $E_{\bar{\psi}}$ is a momentum copy of the encoder updated as $\bar{\psi} \leftarrow \mu\bar{\psi} + (1-\mu)\psi$ with $\mu \in (0, 1)$, and $Z \in \mathbb{R}^{B \times d}$ is the batch of online latents z_t. The variance

and covariance penalties of VICReg [21] are

$$v(Z) = \frac{1}{d}\sum_{j=1}^{d} \max\Big(0,\ \gamma - \sqrt{\mathrm{Var}(Z_{\cdot,j}) + \epsilon}\Big), \tag{9}$$

$$c(Z) = \frac{1}{d}\sum_{i\neq j} \big[C(Z)\big]_{i,j}^2, \qquad C(Z) = \frac{1}{B-1}\sum_{b=1}^{B} \big(Z_{b,\cdot} - \bar{Z}\big)^\top \big(Z_{b,\cdot} - \bar{Z}\big), \tag{10}$$

with margin $\gamma > 0$, numerical constant $0 < \epsilon < \gamma^2$, and batch mean $\bar{Z} \in \mathbb{R}^{1\times d}$. The variance hinge counteracts collapse to a constant embedding and the covariance term counteracts dimensional collapse; the conditions under which the constant encoder is excluded from the global minimizers, and the limits of this guarantee, are analyzed in [12]. During training we track the realized latent scale

$$\bar{\gamma}_n = (1-\alpha_\gamma)\,\bar{\gamma}_{n-1} + \alpha_\gamma \Big(\frac{1}{d}\sum_{j=1}^{d} \mathrm{Var}\left(Z^{(n)}_{\cdot,j}\right) + \epsilon\Big)^{1/2}, \tag{11}$$

where $Z^{(n)}$ is the batch at iteration n and $\alpha_\gamma \in (0,1]$. After this stage E_ψ and P_ω are frozen, which fixes the scale $\bar{\gamma}^\star$ and the latent mean $\bar{\mu}_Z$ used below.

3.3 Primitive Selection and Waypoint Progression

The selector π^h_ϕ is a multilayer perceptron that chooses one of $N_u = 6$ flight primitives $\mathcal{U} = \{\textsc{Cruise}, \textsc{LateralAvoid}, \textsc{VerticalAvoid}, \textsc{YawScan}, \textsc{Approach}, \textsc{BrakeHover}\}$:

$$u_t = \arg\max_{u\in\mathcal{U}} \pi^h_\phi\big(u \mid z_t, g_t, S_t, c_{\text{text}}\big), \tag{12}$$

trained with the cross-entropy loss

$$\mathcal{L}_{\text{CE}}(\phi) = -\mathbb{E}_t\big[\log \pi^h_\phi(u^\star_t \mid z_t, g_t, S_t, c_{\text{text}})\big], \tag{13}$$

where $u^\star_t$ is a label derived from the expert command by fixed rules applied in priority order (BRAKEHOVER if the expert speed is below a hover threshold; APPROACH within a distance of the final waypoint; VERTICALAVOID, LATERALAVOID, or YAWSCAN if the corresponding command component exceeds a threshold; CRUISE otherwise). These rules are a stand-in for annotated or learned segmentations and are not part of the method. The selector output is restricted to $\mathcal{U}$, and BRAKEHOVER doubles as the target of the safety override (Section 3.5).

The active waypoint advances by the symbolic rule

$$m_{t+1} = \begin{cases} m_t + 1, & \text{if } \big\|\hat{p}_t - w^{(m_t)}\big\|_2 < \tau_w \text{ and } m_t < M, \\ m_t, & \text{otherwise,} \end{cases} \tag{14}$$

with acceptance radius $\tau_w > 0$. In [12] the corresponding test is applied to latent distances to goal images, and its soundness depends on a lower co-Lipschitz bound of the encoder that cannot be enforced architecturally. Because flight waypoints are geometric, Eq. (14) tests the state estimate instead: if $\|\hat{p}_t - p_t\|_2 \le \epsilon_p$, a transition implies $\big\|p_t - w^{(m_t)}\big\|_2 < \tau_w + \epsilon_p$ by the triangle inequality, without any assumption on E_ψ.

3.4 Conditional Flow-Matching Policy Solver

The solver conditions on the standardized vector

$$e_t = \mathrm{Concat}\left(\mathrm{Embed}(u_t),\ \tilde{z}_t,\ \tilde{\hat{z}}_{t+1},\ \tilde{S}_t,\ g_t,\ c_{\text{text}}\right) \in \mathbb{R}^{d_e}, \qquad d_e = d_u + 2d + d_s + 4 + 512 = 1073, \tag{15}$$

where $\tilde{z} = (z - \bar{\mu}_Z)/\bar{\gamma}^\star$ is applied to both latents and $\tilde{S}_t$ is S_t standardized with dataset statistics. Actions are mapped to $[-1,1]$ componentwise by the box bounds of $\mathcal{A}_0$; we keep the symbol A_t for the normalized chunk. The vector field $v_\theta : \mathbb{R}^{H\times d_a} \times [0,1] \times \mathbb{R}^{d_e} \to \mathbb{R}^{H\times d_a}$ is a four-block transformer of width 256 that attends over the H knot tokens and a conditioning token. With integration time $\rho \in [0,1]$ and Gaussian noise $\xi \sim \mathcal{N}(0, I)$, $\xi \in \mathbb{R}^{H\times d_a}$, the linear (optimal-transport) path

$$A^{(\rho)}_t = \rho A_t + (1-\rho)\,\xi \tag{16}$$

has constant conditional velocity $A_t - \xi$, and v_θ is trained by the flow-matching regression

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,\,\xi,\,\rho\sim\mathcal{U}[0,1]}\left[\left\| v_\theta\big(A_t^{(\rho)}, \rho \mid e_t\big) - (A_t - \xi)\right\|_F^2\right], \tag{17}$$

where $\|\cdot\|_F$ is the Frobenius norm and the expectation over t runs over the training data. At deployment, a chunk is obtained by integrating $\mathrm{d}A/\mathrm{d}\rho = v_\theta(A, \rho \mid e_t)$ from $A^{(0)} = \xi_t$ with K Euler steps,

$$A^{(\rho_{k+1})} = A^{(\rho_k)} + \frac{1}{K}\, v_\theta\big(A^{(\rho_k)}, \rho_k \mid e_t\big), \qquad \rho_k = \frac{k}{K}, \quad k = 0, \ldots, K-1, \tag{18}$$

which for $K = 2$ reads

$$\hat{A}_t = \xi_t + \tfrac{1}{2} v_\theta(\xi_t, 0 \mid e_t) + \tfrac{1}{2} v_\theta\Big(\xi_t + \tfrac{1}{2} v_\theta(\xi_t, 0 \mid e_t),\, \tfrac{1}{2} \;\Big|\; e_t\Big). \tag{19}$$

The chunk is then denormalized and projected componentwise onto $\mathcal{A}_0$. Although each conditional path in Eq. (16) is straight, the learned marginal field can be curved where the action distribution is multimodal, so the accuracy of two-step integration is an empirical property to be measured (ablation A1, Section 5.7).

Noise persistence. In [12], frequent replanning with fresh noise reduced manipulation success, plausibly because consecutive chunks switched between distinct but individually valid modes. Because the sampler of Eq. (18) is deterministic given ξ_t, we reuse the initial noise across decisions,

$$\xi_t = \begin{cases} \xi_{t-1}, & \text{if } u_t = u_{t-1} \text{ and } t \not\equiv 0 \pmod{N_\xi}, \\ \text{a fresh draw from } \mathcal{N}(0, I), & \text{otherwise,} \end{cases} \tag{20}$$

so that the noise is redrawn when the primitive changes and at least every N_ξ decisions. The following proposition shows what persistence buys.

Proposition 3.1 (Chunk coherence under noise persistence)**.** *Suppose that, on a set containing all Euler iterates, v_θ is L_A-Lipschitz in its first argument and L_e-Lipschitz in its conditioning argument with respect to $\|\cdot\|_F$ and $\|\cdot\|_2$, respectively, with $L_A > 0$. Let $\Phi_K(\xi; e)$ denote the output of Eq. (18) for initial condition ξ and conditioning e. Then, for any ξ and any e, e',*

$$\|\Phi_K(\xi; e) - \Phi_K(\xi; e')\|_F \le \frac{L_e}{L_A}\Big[\big(1 + \tfrac{L_A}{K}\big)^K - 1\Big]\, \|e - e'\|_2 \le \frac{L_e}{L_A}\big(e^{L_A} - 1\big)\, \|e - e'\|_2 . \tag{21}$$

Proof. Let $\delta_k = \big\|A^{(\rho_k)} - A'^{(\rho_k)}\big\|_F$ for the iterates under e and e', so $\delta_0 = 0$. By Eq. (18) and the triangle inequality, $\delta_{k+1} \le (1 + L_A/K)\,\delta_k + (L_e/K)\,\|e - e'\|_2$. Unrolling the recursion gives $\delta_K \le (L_e/K)\,\|e - e'\|_2 \sum_{j=0}^{K-1}(1 + L_A/K)^j = (L_e/L_A)[(1 + L_A/K)^K - 1]\,\|e - e'\|_2$, and $(1 + L_A/K)^K \le e^{L_A}$. □

With a shared ξ, the change between consecutive chunks is therefore bounded in proportion to the change of the conditioning vector, which is small between decisions 33 ms apart; with independent draws no such bound exists, since ξ_t and ξ_{t-1} may be transported to different modes. The constants L_A and L_e are not controlled by training, and dot-product attention is not globally Lipschitz [49], so Eq. (21) explains the mechanism rather than providing a certified bound; its practical effect is tested in ablation A5.

3.5 Epistemic Uncertainty and Velocity Attenuation

At deployment the future latent is unobserved, so the reliability of $\hat{z}_{t+1}$ must be assessed from quantities available at time τ_t. Following deep ensembles [22], we use the disagreement of the heads in Eq. (7), normalized by the frozen latent scale so that it is dimensionless:

$$\sigma_{t+1} = \frac{1}{\sqrt{d}\,\bar{\gamma}^\star}\left(\frac{1}{N_e}\sum_{i=1}^{N_e}\left\|\hat{z}_{t+1}^{(i)} - \hat{z}_{t+1}\right\|_2^2\right)^{1/2}. \tag{22}$$

An asymmetric filter reacts immediately to increases and decays slowly,

$$\bar{\sigma}_{t+1} = \max\big\{\sigma_{t+1},\ (1 - \alpha_\sigma)\,\bar{\sigma}_t + \alpha_\sigma\,\sigma_{t+1}\big\}, \qquad \alpha_\sigma \in (0, 1], \tag{23}$$

which prevents rapid fluctuations of σ_{t+1} from modulating the command at the decision rate. The filtered score governs three mechanisms. First, the translational components of the chunk are attenuated by the monotone, piecewise-linear law

$$\alpha(\bar{\sigma}) = \begin{cases} 1, & \bar{\sigma} \le \sigma_{\mathrm{lo}}, \\ 1 - (1-\alpha_{\min})\dfrac{\bar{\sigma} - \sigma_{\mathrm{lo}}}{\sigma_{\mathrm{crit}} - \sigma_{\mathrm{lo}}}, & \sigma_{\mathrm{lo}} < \bar{\sigma} < \sigma_{\mathrm{crit}}, \end{cases} \tag{24}$$

with $0 < \alpha_{\min} \le 1$ and $0 \le \sigma_{\mathrm{lo}} < \sigma_{\mathrm{crit}}$; α is non-increasing and Lipschitz with constant $(1-\alpha_{\min})/(\sigma_{\mathrm{crit}} - \sigma_{\mathrm{lo}})$, and the yaw rate is not attenuated so that the vehicle can keep reorienting its camera. Second, each chunk carries the validity horizon

$$H_t = \max\Big(H_{\min},\ \big\lfloor H_{\max}\exp(-\beta\,\bar{\sigma}_{t+1})\big\rfloor\Big), \qquad 1 \le H_{\min} \le H_{\max} = H, \tag{25}$$

with sensitivity $\beta > 0$: the executor (Section 3.7) may use the chunk only while its age satisfies $\tau - \tau_t \le H_t\Delta$. In the synchronous execution of [12], H_t is the number of knots executed before replanning; in the asynchronous design adopted here, newer chunks preempt older ones and H_t bounds how stale a chunk may become, for instance when inference stalls, before the fallback is engaged. Third, if $\bar{\sigma}_{t+1} \ge \sigma_{\mathrm{crit}}$, the primitive is overridden, $u_t \leftarrow$ BRAKEHOVER, and the executor performs the braking manoeuvre of Section 3.6.

The thresholds are calibrated on in-distribution validation flights: σ_{lo} is the empirical 0.9-quantile of $\bar{\sigma}$, and σ_{crit} is its $(1-p_{\mathrm{fa}})$-quantile, with p_{fa} chosen such that the expected number of spurious fallbacks per mission, $p_{\mathrm{fa}} f_{\mathrm{dec}} T_{\mathrm{mis}}$, does not exceed a target n_{fa}, where f_{dec} is the decision rate and T_{mis} the mission duration. For $f_{\mathrm{dec}} = 30\,\mathrm{Hz}$, $T_{\mathrm{mis}} = 60\,\mathrm{s}$, and $n_{\mathrm{fa}} = 0.05$, this gives $p_{\mathrm{fa}} \approx 2.8 \times 10^{-5}$, which requires a large validation set because consecutive values of $\bar{\sigma}$ are strongly correlated. The trade-off between safety and conservatism that these choices induce is mapped by ablation A2.

3.6 Braking Envelope and Hover Fallback

Ensemble disagreement is a heuristic: it can remain low on inputs that the frozen encoder maps into familiar regions of $\mathcal{Z}$. We therefore add a model-free bound derived from the depth channel. Let d_t be the smallest range measured in an image region $\mathcal{R}_t$ whose back-projection covers the volume swept by a sphere of radius $r_s \ge r_v$ moving along the commanded direction $\hat{u}_t = v_{t,0}/\left\|v_{t,0}\right\|_2$ out to the sensor range, where $v_{t,k}$ denotes the translational part of $a_{t,k}$. If $\hat{u}_t$ lies outside the field of view, or if the fraction of invalid depth pixels in $\mathcal{R}_t$ exceeds a threshold, the speed is limited to a constant v_{blind}.

Braking profile. When the fallback is engaged at speed v_0 (taken here with zero initial acceleration), the executor commands the velocity profile that minimizes the integrated squared jerk $\int_0^{T_b} \left\|\ddot{v}(\tau)\right\|_2^2 \,\mathrm{d}\tau$ subject to $v(0) = v_0$, $\dot{v}(0) = 0$, $v(T_b) = 0$, and $\dot{v}(T_b) = 0$, with the stopping position left free. The Euler–Lagrange condition $v^{(4)} = 0$ makes the minimizer cubic, and the boundary conditions give

$$v_{\mathrm{brk}}(\tau) = v_0\left(1 - 3\Big(\frac{\tau}{T_b}\Big)^2 + 2\Big(\frac{\tau}{T_b}\Big)^3\right), \qquad T_b = \frac{3\left\|v_0\right\|_2}{2a_{\max}}, \tag{26}$$

whose peak deceleration, attained at $\tau = T_b/2$, equals $1.5\left\|v_0\right\|_2/T_b = a_{\max}$ and whose stopping distance is $\int_0^{T_b}\left\|v_{\mathrm{brk}}\right\|_2 \mathrm{d}\tau = \left\|v_0\right\|_2 T_b/2 = 3\left\|v_0\right\|_2^2/(4a_{\max})$. After the stop, the executor holds the stopping point with a position setpoint, and yaw scanning is permitted so that new observations can lower $\bar{\sigma}$ and release the vehicle.

Speed envelope. Let T_r upper-bound the time from the capture of I_t to the onset of braking if braking becomes necessary; it comprises the worst-case end-to-end latency $T_{\mathrm{e2e}}^{\max}$, the chunk validity $H_t\Delta$ of Eq. (25), and the velocity-tracking lag T_{trk} of the flight controller. With depth error bound ϵ_d, define

$$v_{\mathrm{safe}}(d) = \begin{cases} \dfrac{2a_{\max}}{3}\left(-T_r + \sqrt{T_r^2 + \dfrac{3(d - \epsilon_d - r_s)}{a_{\max}}}\right), & d > \epsilon_d + r_s, \\ 0, & \text{otherwise.} \end{cases} \tag{27}$$

The commanded translational velocity of every knot is scaled by the common factor

$$s_t = \min\left\{\alpha(\bar{\sigma}_{t+1}),\ \frac{v_{\mathrm{safe}}(d_t)}{\max\left(\max_k \left\|v_{t,k}\right\|_2,\ \epsilon_v\right)}\right\}, \qquad a_{t,k}^{\mathrm{cmd}} = \left(s_t\, v_{t,k},\ \dot{\psi}_{t,k}\right), \tag{28}$$

where $\epsilon_v > 0$ guards the division; scaling preserves the direction of every knot.

Assumption 3.1 (Braking envelope)**.** Over the interval from τ_t to the end of a possible braking manoeuvre: (i) the obstacles intersecting the swept volume are static; (ii) every range measured in $\mathcal{R}_t$ exceeds the true range by at most ϵ_d; (iii) the vehicle moves along $\hat{u}_t$ with speed at most $v_{\text{safe}}(d_t)$ until braking begins, and braking begins within T_r of τ_t with zero acceleration; (iv) the flight controller tracks Eq. (26), which requires a deceleration of at most $a_{\max}$.

Proposition 3.2 (Stop within measured free space)**.** *Under Assumption 3.1, the vehicle comes to rest without its center entering the r_s-neighbourhood of any obstacle point imaged in $\mathcal{R}_t$.*

Proof. Let $v \le v_{\text{safe}}(d_t)$ be the speed during the reaction interval. The distance travelled before braking is at most vT_r and the braking distance is $3v^2/(4a_{\max})$ by Eq. (26). For $d_t > \epsilon_d + r_s$, $v_{\text{safe}}(d_t)$ is the positive root of $\frac{3}{4a_{\max}}v^2 + T_r v - (d_t - \epsilon_d - r_s) = 0$, and the left-hand side is increasing in $v \ge 0$, so $vT_r + 3v^2/(4a_{\max}) \le d_t - \epsilon_d - r_s$. By (ii), the true range of every obstacle point in $\mathcal{R}_t$ is at least $d_t - \epsilon_d$; the total travel therefore ends at least r_s short of it. For $d_t \le \epsilon_d + r_s$, $v_{\text{safe}} = 0$ and the vehicle is commanded to hold. □

Remark 3.1. The guarantee is conditional. It does not cover dynamic obstacles, which motivate fast replanning and the uncertainty signal, nor obstacles outside the field of view, which motivate v_{blind}. If braking starts with acceleration a_0 along $\hat{u}_t$, the cubic profile matching $\dot{v}(0) = a_0$ stops after $\|v_0\|_2 T_b/2 + a_0 T_b^2/12$, and the excess can be absorbed into r_s. For illustration, with $a_{\max} = 5\,\text{m/s}^2$, $T_r = 0.4\,\text{s}$, and $d_t - \epsilon_d - r_s = 6\,\text{m}$, Eq. (27) gives $v_{\text{safe}} \approx 5.1\,\text{m/s}$. The envelope is a simplified braking-feasibility constraint in the spirit of control barrier functions [50].

3.7 Receding-Horizon Control Algorithm

Algorithm 1 summarizes the two loops of Aero-VPA. The inference loop runs once per camera frame whenever the previous decision has finished; the executor runs at a fixed rate $f_c = 100\,\text{Hz}$ and evaluates the newest valid chunk at its chunk-local time $\tau - \tau_t$. Because the chunk is indexed by elapsed time since capture rather than by arrival, the knots that correspond to the inference and transport latency are skipped automatically, an approach related to the frozen prefix of real-time chunking [34].

The sequential structure of a decision step is that of [12]: the chain $I_t \to z_t \to u_t \to (\hat{z}_{t+1}, \sigma_{t+1}) \to e_t \to A^{(\rho_1)} \to \cdots \to A^{(\rho_K)}$ contains one evaluation each of E_ψ, π_ϕ^h, and P_ω (whose heads are independent and can be batched) and K evaluations of v_θ. The sequential depth is therefore $D_{\text{VPA}} = K + 3 = 5$ for $K = 2$, independent of H and d_a, and the network time of a decision step is

$$T_{\text{VPA}} = t_E + t_h + t_P + K\, t_v(H), \tag{29}$$

where t_E, t_h, t_P, and $t_v(H)$ are the wall-clock times of the encoder, selector, predictor, and one solver evaluation. The arithmetic work of $t_v(H)$ grows with Hd_a; approximate independence of H holds only while one solver evaluation fits within the available parallelism, which on an Apple M4 was the case up to $H = 64$ [12] and must be re-measured on the target hardware (Section 5.2). The safety computations of Eqs. (22)–(28) add only vector and image-region reductions and no network evaluation.

3.8 Training Procedure

Training follows the staged procedure of [12], adapted to flight. (i) *Representation learning:* E_ψ and P_ω are trained with Eq. (8) on flight logs, then frozen, fixing $\bar{\gamma}^\star$ and $\bar{\mu}_Z$. (ii) *Policy learning:* with latents cached, the selector and solver minimize

$$\mathcal{L}_{\text{pol}}(\phi, \theta) = \mathcal{L}_{\text{CFM}}(\theta) + \lambda_h\, \mathcal{L}_{\text{CE}}(\phi), \tag{30}$$

with weight $\lambda_h > 0$, on expert chunks expressed in the frozen heading frame of their capture time. (iii) *Dataset aggregation:* because imitation from expert states alone suffers from covariate shift, Aero-VPA is rolled out in simulation under a mixture of its own and the expert's commands, the privileged expert relabels the visited states, and stage (ii), optionally also stage (i), is repeated on the aggregated data, following DAgger [51]. The expert is a planner with access to the ground-truth occupied set $\mathcal{X}$, in the manner of [5].

4 Asynchronous System Integration and Edge Deployment

4.1 Compute and Memory Budget on the Jetson Orin Nano

The target module is the Jetson Orin Nano 8 GB with a 1024-core Ampere GPU, 8 GB of LPDDR5 shared by CPU and GPU, and 68 GB/s of memory bandwidth in its original configuration (102 GB/s in the JetPack 6.2 Super configurations) [11]; we use the 15 W power mode. Unlike the Orin NX and AGX Orin, the Orin Nano has no Deep

Algorithm 1 Aero-VPA asynchronous receding-horizon control with safety filter

Input: frozen E_ψ, P_ω, $\bar{\gamma}^\star$, $\bar{\mu}_Z$; trained π_ϕ^h, v_θ; mission G; c_{text} (computed once); $K = 2$

Inference loop (event-driven, one decision per camera frame)

procedure DECIDE(I_t, S_t, τ_t)
  $z_t \leftarrow E_\psi(I_t)$; update m_t by Eq. (14); compute g_t by Eq. (3)
  $u_t \leftarrow \arg\max_u \pi_\phi^h(u \mid z_t, g_t, S_t, c_{\text{text}})$ ▷ Eq. (12)
  $\{\hat{z}_{t+1}^{(i)}\}_{i=1}^{N_e} \leftarrow P_\omega(z_t, \text{Embed}(u_t), \bar{a}_t)$; $\hat{z}_{t+1} \leftarrow$ ensemble mean ▷ Eq. (7)
  compute σ_{t+1} and $\bar{\sigma}_{t+1}$ ▷ Eqs. (22), (23)
  if $\bar{\sigma}_{t+1} \geq \sigma_{\text{crit}}$ **then**
    $u_t \leftarrow$ BRAKEHOVER; REQUESTFALLBACK **and return**
  end if
  update ξ_t by Eq. (20); form e_t by Eq. (15)
  $A \leftarrow \xi_t$
  for $k = 0, \ldots, K-1$ **do** ▷ Eq. (18)
    $A \leftarrow A + \frac{1}{K} v_\theta(A, k/K \mid e_t)$
  end for
  $\hat{A}_t \leftarrow \Pi_{\mathcal{A}_0}(\text{denormalize}(A))$
  $d_t \leftarrow$ minimum valid range in $\mathcal{R}_t$; compute s_t by Eq. (28); H_t by Eq. (25)
  publish chunk $\mathcal{C}_t = (\tau_t, \hat{\psi}_t, \{(s_t v_{t,k}, \dot{\psi}_{t,k})\}_{k=0}^{H-1}, H_t)$ to the buffer
end procedure

Executor (fixed rate $f_c = 100\,Hz$)

procedure TICK(τ)
  $\mathcal{C} \leftarrow$ newest chunk in buffer; $\Delta\tau \leftarrow \tau - \tau_{\mathcal{C}}$
  if fallback requested **or** no chunk **or** $\Delta\tau > H_{\mathcal{C}}\Delta$ **or** odometry stale **then**
    $(v_{\text{ref}}^{\mathcal{W}}, \dot{v}_{\text{ref}}^{\mathcal{W}}) \leftarrow$ braking profile Eq. (26), then hold position
  else
    interpolate knots at $\Delta\tau$ (Eq. (33)); blend with previous chunk (Eq. (35)); rate-limit (Eq. (34))
    rotate to NED: $v_{\text{ref}}^{\mathcal{W}} \leftarrow \mathbf{R}_z(\hat{\psi}_{\mathcal{C}})\, v_{\text{ref}}^{\mathcal{H}}$ ▷ Eq. (32)
  end if
  publish `OffboardControlMode` and `TrajectorySetpoint`$(v_{\text{ref}}^{\mathcal{W}}, \dot{v}_{\text{ref}}^{\mathcal{W}}, \dot{\psi}_{\text{ref}})$
  $\bar{a} \leftarrow$ dispatched setpoint, stored for the next decision
end procedure

Learning Accelerator, so all networks run on the GPU as TensorRT engines. Table 1 lists the configuration and its parameter count, computed analytically from the layer dimensions. Since every evaluation at batch size one reads the weights of the evaluated network, a decision step streams at least

$$M_{\text{dec}} = b\,(|W_E| + |W_h| + |W_P| + K|W_v|) \tag{31}$$

bytes, where $|W_\cdot|$ are the parameter counts of the four networks and $b = 2$ for FP16. For the configuration of Table 1, $M_{\text{dec}} = 43.4\,\text{MB}$, a lower bound of 0.64 ms at 68 GB/s, compared with 88 ms for a single pass of the 3B-parameter decoder discussed in Section 1. At this scale the latency is expected to be dominated by kernel launch overhead and arithmetic (about 1.0 giga multiply-accumulates per decision) rather than by weight streaming, which is why the latency targets of Section 5.5 are hypotheses to be measured rather than consequences of Eq. (31).

4.2 Hierarchical Software Stack

The high level runs on the Jetson under ROS 2 [52]. A camera driver publishes time-stamped, hardware-synchronized RGB and aligned depth frames; a preprocessing node resizes and normalizes them on the GPU; the inference node executes E_ψ, π_ϕ^h, P_ω, and v_θ as TensorRT FP16 engines on a single CUDA stream with preallocated buffers, so that no memory is allocated on the critical path; and the executor node runs on a real-time-prioritized timer at $f_c = 100\,\text{Hz}$. The two loops communicate only through a lock-free single-slot chunk buffer. Each chunk carries the capture stamp τ_t, which the executor uses for latency compensation, so the decision rate $f_{\text{dec}} = \min\{f_{\text{cam}}, 1/T_{\text{inf}}\}$, with camera rate f_{cam} and inference time T_{inf}, can vary without affecting the setpoint rate. A watchdog monitors chunk age, odometry age, and link status and requests the fallback when any of them exceeds its bound.

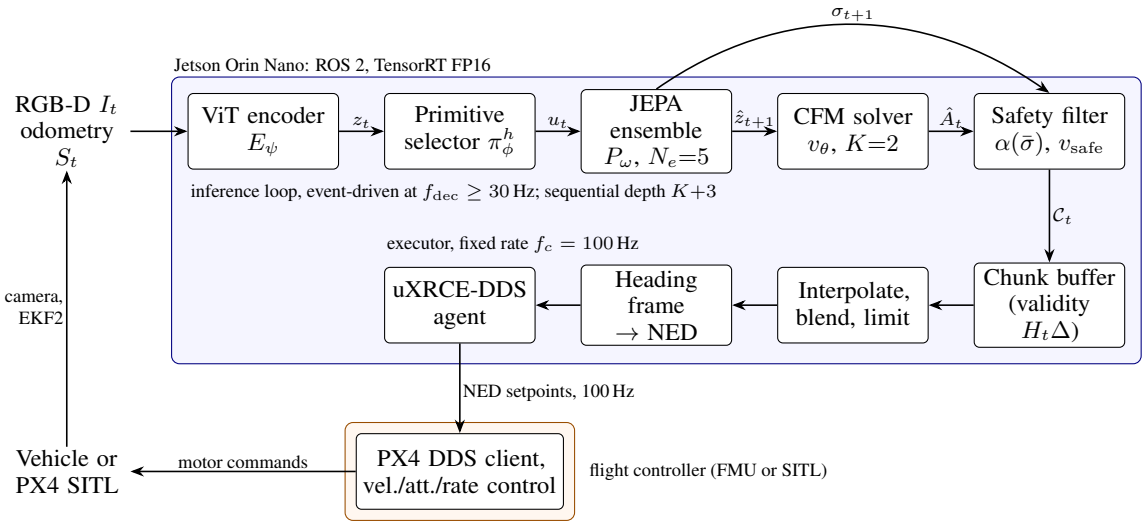


Figure 1: Aero-VPA deployment architecture. The inference loop (top) runs once per camera frame and produces a velocity and yaw-rate chunk $\mathcal{C}_t$ through five sequential network evaluations; the safety filter applies the uncertainty-dependent attenuation $\alpha(\bar{\sigma})$, the braking envelope v_{safe}, and the validity horizon H_t. The executor (bottom) evaluates the newest valid chunk at its chunk-local time, converts it to NED setpoints, and streams them to PX4 over micro-XRCE-DDS; it engages the braking fallback when a fallback is requested or no valid chunk is available.

Table 1: Aero-VPA network configuration and parameter budget (analytic counts for $H = 16$, $d = 256$, $N_u = 6$; FP16 weights). The frozen text encoder is evaluated once per mission and lies outside the per-decision dependency chain.

Module	Configuration	Params	Evals./decision	FP16 weights
Encoder E_ψ	ViT, 4×128×160 input, patch 16, width 384, depth 6, 6 heads	11.17M	1	22.3 MB
Selector π_ϕ^h	MLP $785 \to 512 \to 512 \to 6$	0.67M	1	1.3 MB
Predictor P_ω	5 heads, MLP $292 \to 512 \to 512 \to 256$	2.72M	1 (heads batched)	5.4 MB
Solver v_θ	Transformer, width 256, depth 4, 4 heads, H+1 = 17 tokens	3.57M	$K = 2$	7.1 MB
Trainable total		18.13M	K+3 = 5 sequential	36.3 MB
Text encoder (frozen)	CLIP ViT-B/32 text tower	63.4M	once per mission	–

4.3 PX4 Interface and Minimum-Acceleration Setpoint Interpolation

The low level is a PX4 flight controller [46] (e.g., an FMUv6X-class Pixhawk) connected to the Jetson through the uXRCE-DDS bridge, in which a client on PX4 exchanges uORB topics with an agent on the companion computer over a serial link (e.g., 921 600 baud) or a dedicated Ethernet link [23]. In offboard mode, PX4 requires a continuous proof-of-life stream of at least 2 Hz, provided by `OffboardControlMode`, and triggers its configured failsafe if the stream is interrupted for longer than `COM_OF_LOSS_T` (default 1.0 s) [20]; we publish both this message and `TrajectorySetpoint` at $f_c = 100$ Hz, so that PX4's failsafe acts only as the last of three layers behind the executor fallback and the watchdog. `TrajectorySetpoint` is expressed in the local NED frame, with unused fields set to NaN, so the heading-frame velocities of a chunk are rotated with the yaw frozen at its capture time,

$$v_{\text{ref}}^{\mathcal{W}}(\tau) = \mathbf{R}_z(\hat{\psi}_t)\, v_{\text{ref}}^{\mathcal{H}_t}(\tau), \qquad \dot{\psi}_{\text{ref}}(\tau) = \dot{\psi}_{\text{ref}}^{\mathcal{H}_t}(\tau), \tag{32}$$

where the yaw rate maps directly onto the NED `yawspeed` field because the down axes of $\mathcal{H}_t$ and $\mathcal{W}$ coincide.

Interpolation. The executor runs faster than the knot rate $1/\Delta$ and also supplies an acceleration feedforward, which PX4 expects to be kinematically consistent with the velocity setpoint. Among all absolutely continuous velocity references that pass through the knots, $v_{\text{ref}}(k\Delta) = v_{t,k}$, the one minimizing the integrated squared acceleration $\int \|\dot{v}_{\text{ref}}(\tau)\|_2^2 \, \mathrm{d}\tau$ satisfies $\ddot{v}_{\text{ref}} = 0$ between knots by the Euler–Lagrange equation and is therefore piecewise linear:

$$v_{\text{ref}}(\tau) = v_{t,k} + \frac{\tau - k\Delta}{\Delta}\big(v_{t,k+1} - v_{t,k}\big), \qquad \dot{v}_{\text{ref}}(\tau) = \frac{v_{t,k+1} - v_{t,k}}{\Delta}, \qquad \tau \in [k\Delta, (k+1)\Delta), \tag{33}$$

with τ measured from the capture time τ_t; the yaw rate is interpolated in the same way. The acceleration feedforward is piecewise constant and exactly consistent with the velocity reference. To keep it feasible, consecutive knots are rate-limited before interpolation,

$$v_{t,k+1} \leftarrow v_{t,k} + \mathrm{clip}_{a_{\max}\Delta}\left(v_{t,k+1} - v_{t,k}\right), \qquad \mathrm{clip}_c(y) = y\,\min\left(1, c/\left\|y\right\|_2\right), \tag{34}$$

which bounds $\left\|\dot{v}_{\mathrm{ref}}\right\|_2$ by $a_{\max}$.

Chunk blending. When a new chunk $\mathcal{C}_{t'}$ replaces $\mathcal{C}_t$ at time τ_s, the executor cross-fades the two references over a window T_{bl},

$$v_{\mathrm{ref}}(\tau) = \lambda(\tau)\, v_{\mathrm{ref}}^{(t')}(\tau) + \big(1 - \lambda(\tau)\big)\, v_{\mathrm{ref}}^{(t)}(\tau), \qquad \lambda(\tau) = \min\Big(1, \frac{\tau - \tau_s}{T_{\mathrm{bl}}}\Big), \tag{35}$$

where both references are expressed in $\mathcal{W}$ and evaluated at their own chunk-local times. Since $\lambda(\tau_s) = 0$ and both references are continuous, v_{ref} is continuous at τ_s; if $\mathcal{C}_t$ expires within the window, its last knot is held. The acceleration feedforward of the blended reference is the corresponding derivative of Eq. (35). Inpainting-based real-time chunking [34] is an alternative evaluated in ablation A8.

Link budget. A `TrajectorySetpoint` carries 64 bytes of payload. Assuming at most 128 bytes per serialized message including framing, the setpoint stream and an odometry stream of equal rate occupy about 28% of the 92 160 B/s capacity of a 921 600-baud 8N1 link at 100 Hz each; the dedicated Ethernet link removes this constraint.

4.4 Latency Budget

We decompose the end-to-end latency from photon capture to the setpoint received by PX4 as

$$T_{\mathrm{e2e}} = t_{\mathrm{cap}} + t_{\mathrm{pre}} + \underbrace{t_E + t_h + t_P + K\, t_v(H)}_{T_{\mathrm{VPA}}} + t_{\mathrm{safe}} + t_{\mathrm{exec}} + t_{\mathrm{dds}}, \tag{36}$$

where t_{cap} is the exposure, readout, and driver delay, t_{pre} the preprocessing time, t_{safe} the time of the safety computations, $t_{\mathrm{exec}} \le 1/f_c$ the wait until the next executor tick, and t_{dds} the transport time to the PX4 client. We do not adopt a numerical value for t_{dds} from the literature; every term of Eq. (36) is measured in the HITL protocol (Section 5.2), and the measured worst case $T_{\mathrm{e2e}}^{\max}$ enters T_r in Eq. (27).

4.5 Cross-Embodiment Adaptation to Quadrupeds

The architecture transfers to legged robots through the same separation of time scales. For a quadruped (Legged-VPA), the chunk consists of planar base twists, elements of $\mathfrak{se}(2)$,

$$A_t^{\mathrm{leg}} = [v_x, v_y, \omega_z]_{t:t+H-1} \in \mathbb{R}^{H\times 3}, \tag{37}$$

where v_x, v_y are base-frame linear velocities and ω_z is the yaw rate. Because leg phase and contact state make single-step proprioception non-Markovian, the proprioceptive input is a history of $L = 30$ steps of joint positions $q_j \in \mathbb{R}^{12}$, joint velocities $\dot{q}_j \in \mathbb{R}^{12}$, and projected gravity $n^{\mathcal{B}} \in \mathbb{R}^3$,

$$S_t^{\mathrm{leg}} = \left(q_{j,t-\ell},\, \dot{q}_{j,t-\ell},\, n_{t-\ell}^{\mathcal{B}}\right)_{\ell=0}^{L-1} \in \mathbb{R}^{L(12+12+3)} = \mathbb{R}^{810}, \tag{38}$$

which a linear layer projects before it enters e_t. The executor samples Eq. (33) at 50 Hz and passes the twist to a low-level locomotion policy trained by reinforcement learning, such as Walk These Ways [45], whose joint targets are tracked by a proportional-derivative controller at 200 Hz. The safety layer carries over with $\alpha(\bar{\sigma})$ applied to (v_x, v_y), the braking envelope recomputed with the deceleration limit of the gait, and BRAKEHOVER replaced by a stop-and-stand primitive. Whether E_ψ and P_ω can be shared across aerial and legged embodiments is an open question: it requires pretraining data from both, and camera height, ego-motion, and motion blur differ substantially between them. We treat sharing as a hypothesis, whereas v_θ and the executor are embodiment-specific by construction.

5 Experimental Design and Benchmark Protocol

Status. None of the experiments in this section has been executed. The section specifies the environments, data, hardware, baselines, metrics, hypotheses, and statistical analysis before execution, so that the eventual results can be judged against criteria fixed in advance. Target values in Tables 2 and 3 are acceptance criteria, not results.

5.1 Simulation Setup

Platforms. The primary environment is PX4 SITL coupled to Gazebo [53] in its Harmonic long-term-support release (Gazebo Garden reached end of life in November 2024 and is not supported by current PX4 releases), using the PX4 quadrotor model with a front-facing depth camera (`gz_x500_depth`) augmented with an aligned RGB stream. For photorealistic tests of visual robustness, the same vehicle is simulated in NVIDIA Isaac Sim [54] with PX4 SITL attached through a bridge such as the Pegasus Simulator [55]. A single PX4 release is pinned for all runs and recorded with the results.

Environment classes. Worlds are generated procedurally from integer seeds. (E1) *Dense forest*: trees placed by Poisson-disc sampling at three densities, with trunk diameters and canopy heights drawn from fixed distributions. (E2) *Collapsed indoor structure*: rooms and corridors with rubble, tilted slabs, hanging cables, and openings whose minimum width ranges from 1.5 to 3 times the vehicle diameter, under low and non-uniform illumination. (E3) *Dynamic clutter*: E1 and E2 with moving obstacles crossing the nominal path at speeds of 0.5–2 m/s. Missions consist of 3–6 waypoints with straight-line spacing of 10–40 m. Each episode runs at a commanded speed limit $\bar{v}_h \in \{3, 5, 7\}$ m/s.

Perturbations. Physical perturbations comprise steady wind and gusts up to 5 m/s injected mid-flight through the simulator's wind model. Visual perturbations comprise illumination changes, unseen textures, moving background elements, motion blur, and depth noise and dropout following a range-dependent stereo error model. Training uses domain randomization [56] over a subset of these factors; held-out factors define the out-of-distribution (OOD) test set.

Splits. Training, validation, and test worlds use disjoint seed ranges. The test set comprises 10 held-out worlds per environment class and difficulty level, an in-distribution (ID) condition, and an OOD condition with held-out visual factors and wind.

Data. Demonstrations are produced by a privileged expert that plans on the ground-truth occupancy and tracks the resulting minimum-snap trajectories [3]; expert chunks are recorded at the knot rate $1/\Delta = 50$ Hz together with the RGB-D stream and EKF2 estimates. We plan 2,000 expert flights over 200 training worlds followed by three DAgger rounds of 500 flights each (Section 3.8); dataset sizes and the expert's own success rate are reported.

5.2 Hardware-in-the-Loop Specification

Configuration. The trained pipeline is exported to ONNX and compiled with TensorRT in FP16 (INT8 with a calibration cache as ablation A9) on a Jetson Orin Nano 8 GB developer kit in the 15 W mode, with clock settings fixed and reported. A simulation host with a discrete GPU runs PX4 SITL and the simulator; rendered RGB-D frames are streamed to the Jetson over a dedicated Gigabit Ethernet link, and the uXRCE-DDS agent on the Jetson connects to the SITL client over UDP. In a second stage, PX4 runs on a physical flight controller in PX4's HITL mode, connected to the Jetson over a serial link at 921 600 baud, so that the serial transport and flight-controller scheduling are exercised.

Timing validity. Host and Jetson clocks are synchronized with the Precision Time Protocol. The simulator runs in real time, and episodes whose real-time factor falls below 0.98 at any point are discarded and reported. The image transport from host to Jetson is not present on a vehicle; its latency is measured separately and reported both included in and excluded from T_{e2e}.

Instrumentation. Every term of Eq. (36) is logged per decision from synchronized timestamps: capture, preprocessing, the four network times (CUDA events), safety filter, executor wait, and DDS transport (round trip halved and cross-checked against synchronized one-way stamps). Power and utilization are logged with `tegrastats` at 100 ms intervals, including module input power (VDD_IN), the combined CPU–GPU–CV rail (VDD_CPU_GPU_CV), GPU and memory-controller utilization, and temperatures. On this module the GPU is not metered on a separate rail, so the combined rail is reported as the GPU-power metric.

5.3 Baselines

All baselines receive the same observations (RGB-D, EKF2 state, mission waypoints), run on the same Jetson in the same power mode, and command PX4 through the same interface with the same speed limits; imitation-learning baselines use the same expert data and DAgger budget, and hyperparameters are tuned with equal search budgets on validation worlds.

- **B1, classical:** VINS-Fusion [36] stereo-inertial odometry, an OctoMap [2] at 0.1 m resolution, A* search on the occupancy grid, and minimum-snap trajectory generation [3] tracked through PX4 position setpoints.
- **B2, end-to-end reinforcement learning:** vision-based PPO [57] and SAC [58] policies with an encoder of the same size as E_ψ and an asymmetric critic with privileged state, trained on the reward of Eq. (5) with a progress term in a GPU-parallel simulator [59] with matched vehicle parameters and depth rendering, and evaluated in the common SITL protocol. The training budget is reported in environment steps.
- **B3, generative policies:** a downscaled Diffusion Policy [19] with the encoder of Table 1, a solver of equal size, and DDIM sampling with 10 and 100 denoising steps; Octo-Small (27M parameters) [33], fine-tuned with a four-dimensional velocity action head; and Qantara [35], trained from scratch on the same data with its published objective, with RGB-D input and the waypoint feature g_t of Eq. (3) added to its conditioning so that it receives the same mission information as the other methods. Qantara is evaluated through its behaviour-cloning and video-inverse paths with two Euler steps; its latent-planning path is excluded from the real-time protocol because its reported planning time of 0.4–1.2 s per step on an H100 GPU exceeds the control period by more than an order of magnitude.

5.4 Evaluation Metrics

For N episodes, let $\mathcal{E}_i^{\mathrm{succ}}$ denote the event that episode i reaches the final waypoint within $\tau_w = 1$ m before the time limit $T_{\max}$ without collision, and $\mathcal{E}_i^{\mathrm{col}}$ the event that $\mathrm{dist}(p(\tau), \mathcal{X}) < r_v$ at some time τ during the episode. The primary metrics are

$$\mathrm{SR} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\big[\mathcal{E}_i^{\mathrm{succ}}\big], \qquad \mathrm{CR} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\big[\mathcal{E}_i^{\mathrm{col}}\big], \tag{39}$$

$$\bar{v} = \frac{1}{|\mathcal{I}_{\mathrm{succ}}|}\sum_{i\in\mathcal{I}_{\mathrm{succ}}} \frac{\ell_i}{T_i}, \tag{40}$$

$$T_{\mathrm{e2e}}^{(q)} = \text{the } q\text{-quantile of } \{T_{\mathrm{e2e},t}\}_t, \quad q \in \{0.5, 0.95, 0.99\}, \tag{41}$$

$$\bar{P}_{\mathrm{rail}} = \frac{1}{T}\int_0^T P_{\mathrm{rail}}(\tau)\,\mathrm{d}\tau, \qquad E_{\mathrm{dec}} = \frac{\bar{P}_{\mathrm{VDD_IN}}}{f_{\mathrm{dec}}}, \tag{42}$$

where $\mathcal{I}_{\mathrm{succ}}$ is the set of successful episodes, ℓ_i and T_i are the flown path length and duration of episode i, $T_{\mathrm{e2e},t}$ is the latency of Eq. (36) for decision t, P_{rail} is the power of the rail VDD_CPU_GPU_CV or VDD_IN over a logging window of length T, and E_{dec} is the module energy per decision. Secondary metrics are the decision rate f_{dec}; the deadline-miss rate, i.e., the fraction of executor ticks served by the fallback because no valid chunk was available; the root-mean-square jerk of the flown trajectory; the magnitude of velocity-reference discontinuities at chunk switches before blending; and, for the safety layer, the area under the receiver operating characteristic curve

$$\mathrm{AUROC} = \Pr\left(\bar{\sigma}_{t^+} > \bar{\sigma}_{t^-}\right), \tag{43}$$

where t^+ and t^- are decision steps drawn uniformly from those followed and not followed, respectively, by a collision or a near-miss within $T_h = 1$ s, together with the rates of true and spurious fallbacks and the mean speed reduction caused by s_t in perturbation-free segments, which quantifies conservatism.

5.5 Hypotheses and Statistical Protocol

Hypothesis 1 (Latency)**.** On the Jetson Orin Nano in the 15 W mode with TensorRT FP16, T_{VPA} for $K = 2$ and $H = 16$ has a 99th percentile of at most 20 ms, and $T_{\mathrm{e2e}}^{(0.99)}$ excluding the simulator image transport is at most 50 ms; $t_v(H)$ remains within 10% of its value at $H = 8$ for $H \le 64$.

Hypothesis 2 (Task performance)**.** At each speed limit, the SR of Aero-VPA is non-inferior to that of the best B3 baseline with a margin of 5 percentage points, while its $T_{\mathrm{e2e}}^{(0.95)}$ is lower; and its SR decreases less than that of B1 between $\bar{v}_h = 3$ and 7 m/s.

Hypothesis 3 (Uncertainty and safety)**.** Under OOD conditions, $\bar{\sigma}$ predicts collisions and near-misses within 1 s with $\mathrm{AUROC} \ge 0.75$, and the full safety layer reduces CR relative to the unfiltered policy while increasing the mean mission time by at most 15%.

Hypothesis 4 (Representation)**.** Under held-out visual perturbations, the relative SR decrease of Aero-VPA is smaller than that of an otherwise identical policy whose encoder is trained with a pixel-reconstruction objective.

Hypothesis 5 (Chunk coherence)**.** Noise persistence (Eq. (20)) reduces the median velocity-reference discontinuity at chunk switches and does not reduce SR in environment class E3.

Table 2: Target comparison schema (template; no results). Each block is reported per environment class (E1–E3), condition (ID/OOD), and speed limit $\bar{v}_h \in \{3, 5, 7\}$ m/s. "–" denotes a quantity to be measured under the protocol of Section 5; $\bar{P}_{\mathrm{CGC}}$ is the mean power of the VDD_CPU_GPU_CV rail. The last row states the acceptance criteria for Aero-VPA (V1) and the hypothesis each criterion tests.

ID	Method	Params	SR (%)	CR (%)	$\bar{v}$ (m/s)	T_{e2e} p50 / p95 (ms)	$\bar{P}_{\mathrm{CGC}}$ (W)	E_{dec} (mJ)
B1	VINS-Fusion + OctoMap + min-snap	–	–	–	–	– / –	–	–
B2a	Vision PPO	matched	–	–	–	– / –	–	–
B2b	Vision SAC	matched	–	–	–	– / –	–	–
B3a	Diffusion Policy (DDIM 10 / 100)	matched	–	–	–	– / –	–	–
B3b	Octo-Small, fine-tuned	27M	–	–	–	– / –	–	–
B3c	Qantara (behaviour cloning / video-inverse)	≈22M	–	–	–	– / –	–	–
V0	Aero-VPA without safety layer	18.13M	–	–	–	– / –	–	–
V1	Aero-VPA, full ($K = 2$)	18.13M	–	–	–	– / –	–	–
Acceptance criteria for V1 (hypothesis tested)			$\geq$ B3* − 5 (H2)	$<$ V0 (H3)	report vs. B1	p99 $\leq$ 50 (H1)	15 W mode	report

B3*: best B3 baseline at the same speed limit (non-inferiority margin 5 percentage points). Network-only p99 target: $T_{\mathrm{VPA}} \leq 20$ ms.

Statistical analysis. Each method is trained with three seeds. For each seed, environment class, condition, and speed limit, 500 episodes are run (10 test worlds × 50 start states), with identical world seeds and start states across methods. Proportions are reported with 95% Wilson intervals and continuous metrics with bootstrap intervals over episodes, together with seed-level means and standard deviations. Paired binary outcomes are compared with McNemar's exact test, and the non-inferiority test of Hypothesis 2 uses the lower bound of a one-sided 95% interval on the paired SR difference; p-values within each hypothesis family are corrected with the Holm–Bonferroni procedure. For an unpaired two-sided two-proportion z-test at significance level α_s and power $1 - \beta_s$, the number of episodes per method needed to detect a difference between success probabilities p_1 and p_2 is

$$n = \frac{\left(z_{1-\alpha_s/2}\sqrt{2\bar{p}(1-\bar{p})} + z_{1-\beta_s}\sqrt{p_1(1-p_1)+p_2(1-p_2)}\right)^2}{(p_1-p_2)^2}, \qquad \bar{p} = \frac{p_1+p_2}{2}, \tag{44}$$

where z_q is the standard normal q-quantile. For $p_1 = 0.6$, $p_2 = 0.7$, $\alpha_s = 0.05$, and $1 - \beta_s = 0.8$, Eq. (44) gives $n \approx 356$; with 500 episodes the power is about 0.91, and the paired design is expected to increase it further.

5.6 Target Comparison Schema

Table 2 is the template in which the main comparison will be reported, for each environment class, condition, and speed limit. All measured cells are empty by design.

5.7 Ablation Matrix

Ablations vary one factor at a time around the default configuration (V1), except A2, which is a full grid because its purpose is to trace the safety–conservatism Pareto front in the plane of collision rate and normalized mission time. Table 3 lists the factors, levels, readouts, and the hypothesis each ablation addresses.

6 Discussion and Future Work

6.1 Expected Engineering Hurdles

Sim-to-real gap in depth and appearance. Active stereo depth degrades outdoors when sunlight overwhelms the projector, on thin structures such as branches and cables, and at range, where the error grows roughly quadratically with distance. These effects bias both the encoder input and the braking envelope through ϵ_d. We plan range-dependent noise and dropout models in training, a conservative range-dependent ϵ_d in Eq. (27), and fine-tuning of the stage-(ii) modules on a small amount of real flight data while keeping E_ψ frozen.

Aerodynamic effects. Ground effect near the floor and rubble [39], wall and ceiling effects in narrow openings, rotor downwash recirculating in confined rooms, and gusts [40] all perturb velocity tracking. They enlarge T_{trk} and may

Table 3: Planned ablation matrix. Defaults are underlined; all other factors are held at their defaults. "Pareto" denotes the front of collision rate versus mission time normalized by that of V0.

ID	Factor	Levels	Primary readouts	Tests
A1	CFM integration steps	$K \in \{1, \underline{2}, 4, 8\}$	Kt_v, T_{VPA}, SR, CR, chunk error vs. $K = 64$ reference	H1, H2
A2	Uncertainty threshold $\times$ attenuation floor	$p_{\mathrm{fa}} \in \{10^{-2}, 10^{-3}, \underline{2.8{\times}10^{-5}}\}$ or fallback off; $\times\ \alpha_{\min} \in \{0.25, \underline{0.5}, 1\}$	Pareto, spurious and true fallback rates	H3
A3	Attenuation law	none, <u>piecewise linear (Eq. (24))</u>, exponential	CR, $\bar{v}$, jerk	H3
A4	Braking envelope	off, <u>on</u>	CR in E1/E2, $\bar{v}$, conservatism	H3
A5	Noise persistence	fresh noise, <u>persistent, $N_\xi = 30$</u>, persistent, $N_\xi = 10$	discontinuity at switches, SR in E3	H5
A6	Latent predictor	removed (reactive CFM), <u>JEPA + VICReg</u>, JEPA without $v(Z), c(Z)$	SR, effective rank of $C(Z)$, AUROC	H4
A7	Encoder objective	<u>JEPA</u>, pixel reconstruction, frozen pretrained DINOv2-S [60]	relative SR drop under visual OOD	H4
A8	Execution mode	<u>asynchronous + blending</u>, asynchronous + inpainting [34], synchronous prefix with fixed $H_t \in \{4, 8, 16\}$, synchronous adaptive H_t	SR, deadline-miss rate, discontinuity	H5
A9	Numerical precision	FP32, <u>FP16</u>, INT8	T_{VPA}, E_{dec}, chunk deviation from FP32	H1
A10	Chunk horizon	$H \in \{8, \underline{16}, 32, 64\}$	$t_v(H)$, SR, jerk	H1

violate Assumption 3.1(iv), so the tracking lag and the attainable deceleration will be identified on the physical airframe and entered into T_r and $a_{\max}$ with margins, and the flight envelope will be restricted near surfaces.

Embedded execution. Converting attention layers to FP16 can overflow or reorder reductions; we will verify the parity of the TensorRT engines with the PyTorch reference to a fixed tolerance on recorded inputs before any flight. In an enclosed airframe, the module may throttle thermally even within the 15 W mode, which changes T_{VPA} over a mission; temperature and clock logging during long HITL runs will determine whether active cooling is required. Rolling-shutter distortion and motion blur at speed alter the input distribution in ways that simulation reproduces only approximately.

6.2 Limitations of the Design

The ensemble of Eq. (22) measures disagreement among predictor heads that share a frozen encoder. It therefore captures predictor uncertainty but can miss failures of the encoder itself: an unfamiliar image mapped to a familiar latent yields confident, agreeing heads. The braking envelope partially mitigates this risk for static obstacles; latent-density scores and input-space OOD detectors are candidate complements to be compared within ablation A6. The guarantees stated here are conditional: Proposition 3.1 depends on Lipschitz constants that are not controlled, and Proposition 3.2 depends on static obstacles, bounded depth error, and accurate tracking. The primitive dictionary bounds the repertoire of behaviours, and the rule-based labels of Section 3.3 may segment expert behaviour imperfectly. Finally, the uncertainty-adaptive horizon did not improve success in the manipulation experiments of [12]; whether it is beneficial in dynamic flight, where it acts as a staleness bound rather than an execution prefix, is an open question that ablation A8 is designed to answer.

6.3 Planned Physical Flight Trials

Physical trials will proceed only if the HITL stage meets Hypothesis 1 and the safety layer meets Hypothesis 3 in simulation. They are staged as follows: (i) tethered and then free flight in a netted indoor arena with motion-capture ground truth, with the speed limit raised from 1 to 3 and then 5 m/s only after each level meets its criteria; (ii) static and moving foam obstacles in the arena to test the braking envelope and the fallback, including deliberately induced inference stalls and link interruptions; and (iii) outdoor flight in sparse and then dense forest with RTK-GNSS ground truth. Throughout, a safety pilot holds a radio override, the arena or field is geofenced, PX4's offboard-loss failsafe remains active, and every flight is logged with the instrumentation of Section 5.2.

6.4 Concluding Remarks

Aero-VPA replaces autoregressive action decoding with a short, fixed chain of small networks: a non-reconstructive JEPA world model, a discrete primitive selector, and a two-step flow-matching solver that produces a full velocity chunk in five sequential evaluations. It couples this chain to a safety layer driven by latent ensemble disagreement and by a model-free braking envelope, and to an asynchronous executor that meets the timing contract of the PX4 offboard interface. We have stated which properties follow from the architecture, which depend on assumptions about the learned components or the environment, and how each will be tested. The benchmark protocol is fixed before execution, and its results, positive or negative, will determine whether the latency advantage of the approach translates into safer and faster flight.

References

[1] Tong Qin, Peiliang Li, and Shaojie Shen. VINS-Mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4):1004–1020, 2018.

[2] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. OctoMap: An efficient probabilistic 3D mapping framework based on octrees. *Autonomous Robots*, 34(3):189–206, 2013.

[3] Daniel Mellinger and Vijay Kumar. Minimum snap trajectory generation and control for quadrotors. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2520–2525, 2011.

[4] Xin Zhou, Zhepei Wang, Hongkai Ye, Chao Xu, and Fei Gao. EGO-Planner: An ESDF-free gradient-based local planner for quadrotors. *IEEE Robotics and Automation Letters*, 6(2):478–485, 2021.

[5] Antonio Loquercio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Learning high-speed flight in the wild. *Science Robotics*, 6(59):eabg5810, 2021.

[6] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature*, 620:982–987, 2023.

[7] Yunlong Song, Angel Romero, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Reaching the limit in autonomous racing: Optimal control versus reinforcement learning. *Science Robotics*, 8(82):eadg1462, 2023.

[8] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[9] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[10] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[11] NVIDIA. NVIDIA JetPack 6.2 brings Super Mode to NVIDIA Jetson Orin Nano and Jetson Orin NX modules. NVIDIA Technical Blog, https://developer.nvidia.com/blog/nvidia-jetpack-6-2-brings-super-mode-to-nvidia-jetson-orin-nano-and-jetson-orin-nx-modules/, 2025. Accessed: 2026-10-25.

[12] Yeonseok Lee. Vision-Predictor-Action: Towards efficient embodied AI via neuro-symbolic JEPA predictors. *arXiv preprint arXiv:2610.15556*, 2026.

[13] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, version 0.9.2, 2022.

[14] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

[15] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.

[16] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[17] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

[18] Michael S. Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023.

[19] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10–11):1684–1704, 2025.

[20] PX4 Autopilot. Offboard mode (generic/all frames). PX4 User Guide, https://docs.px4.io/main/en/flight_modes/offboard, 2026. Accessed: 2026-10-25.

[21] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022.

[22] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[23] PX4 Autopilot. uXRCE-DDS (PX4–ROS 2/DDS bridge). PX4 User Guide, https://docs.px4.io/main/en/middleware/uxrce_dds, 2026. Accessed: 2026-10-25.

[24] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025.

[25] Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[26] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.

[27] Aleksandar Vujinovic and Aleksandar Kovačević. ACT-JEPA: Novel joint-embedding predictive architecture for efficient policy representation learning. *IEEE Access*, 2026.

[28] Jingwen Sun, Wenyao Zhang, Zekun Qi, Shaojie Ren, Zezhi Liu, Hanxin Zhu, Guangzhong Sun, Xin Jin, and Zhibo Chen. VLA-JEPA: Enhancing vision-language-action model with latent world model. *arXiv preprint arXiv:2602.10098*, 2026.

[29] Yeonseok Lee. NS-JEPA: Towards efficient embodied AI via neuro-symbolic joint-embedding predictive architecture. Zenodo, https://doi.org/10.5281/zenodo.22493915, 2026.

[30] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[31] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023.

[32] Aaditya Prasad, Kevin Lin, Jimmy Wu, Linqi Zhou, and Jeannette Bohg. Consistency policy: Accelerated visuomotor policies via consistency distillation. In *Robotics: Science and Systems (RSS)*, 2024.

[33] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.

[34] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.07339.

[35] Ruslan Rakhimov, George Bredis, Yuriy Maksyuta, and Daniil Gavrilov. Qantara: Bridge-flow training for multi-paradigm JEPA inference. *arXiv preprint arXiv:2607.04978*, 2026.

[36] Tong Qin, Jie Pan, Shaozu Cao, and Shaojie Shen. A general optimization-based framework for local odometry estimation with multiple sensors. *arXiv preprint arXiv:1901.03638*, 2019.

[37] Xin Zhou, Xiangyong Wen, Zhepei Wang, Yuman Gao, Haojia Li, Qianhao Wang, et al. Swarm of micro flying robots in the wild. *Science Robotics*, 7(66):eabm5954, 2022.

[38] Philipp Foehn, Elia Kaufmann, Angel Romero, Robert Penicka, Sihao Sun, Leonard Bauersfeld, et al. Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight. *Science Robotics*, 7(67):eabl6259, 2022.

[39] Guanya Shi, Xichen Shi, Michael O'Connell, Rose Yu, Kamyar Azizzadenesheli, Animashree Anandkumar, Yisong Yue, and Soon-Jo Chung. Neural lander: Stable drone landing control using learned dynamics. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9784–9790, 2019.

[40] Michael O'Connell, Guanya Shi, Xichen Shi, Kamyar Azizzadenesheli, Anima Anandkumar, Yisong Yue, and Soon-Jo Chung. Neural-Fly enables rapid learning for agile flight in strong winds. *Science Robotics*, 7(66):eabm6597, 2022.

[41] Dhruv Shah, Ajay Sridhar, Arjun Bhorkar, Noriaki Hirose, and Sergey Levine. GNM: A general navigation model to drive any robot. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[42] Ajay Sridhar, Dhruv Shah, Catherine Glossop, and Sergey Levine. NoMaD: Goal masked diffusion policies for navigation and exploration. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[43] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47):eabc5986, 2020.

[44] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Robotics: Science and Systems (RSS)*, 2021.

[45] Gabriel B. Margolis and Pulkit Agrawal. Walk these ways: Tuning robot control for generalization with multiplicity of behavior. In *Conference on Robot Learning (CoRL)*, 2022.

[46] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240, 2015.

[47] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[48] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.

[49] Hyunjik Kim, George Papamakarios, and Andriy Mnih. The Lipschitz constant of self-attention. In *International Conference on Machine Learning (ICML)*, pages 5562–5571, 2021.

[50] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *European Control Conference (ECC)*, pages 3420–3431, 2019.

[51] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.

[52] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.

[53] Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2149–2154, 2004.

[54] NVIDIA. Isaac Sim. https://developer.nvidia.com/isaac/sim, 2026. Accessed: 2026-10-25.

[55] Marcelo Jacinto, João Pinto, Jay Patrikar, et al. Pegasus Simulator: An Isaac Sim framework for multiple aerial vehicles simulation. In *International Conference on Unmanned Aircraft Systems (ICUAS)*, 2024.

[56] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.

[57] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[58] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.

[59] Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, et al. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023.

[60] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.